

NEXUS

Networked Engine for eXplainable, Unified commerce & Settlement

Razorpay AI Buildathon — Track 01: AI Growth & Agentic Commerce

NEXUS is a commerce engine that lets both humans and AI agents shop, decide, and transact with a merchant — with every money action explainable, bounded, gated, and logged.

Builder: Varun Kumar (VS7)

Deadline: September 5, 2026 | Build window: 10 days

Prepared: August 23, 2026

Category	Answer
What	One commerce engine, two front doors: a human buyer via chat, and an AI buyer agent via a structured API.
Why	Track 01 rewards agent-to-agent commerce readiness. Almost no other applicant will build this side of the brief.
How	Catalog + agent reasoning + gate + audit log — one shared core, used by both entry paths — built on Razorpay's test-mode APIs.
Output	A working public repo, an architecture diagram, and a 5-minute demo video showing a human AND an AI agent both completing a transaction.

This document is the locked project brief — scope, architecture, stack, and timeline for the 10-day build. It is written so anyone unfamiliar with the project can read it start to finish and understand exactly what is being built and why.

1 What We Are Building

A **conversational and agent-callable commerce engine** — NEXUS — for a merchant on Razorpay. It lets a product be discovered, recommended, upsold, and purchased — by a **human** typing in a chat window, or by **another AI agent** calling a structured API. Both paths run through the exact same decision engine underneath.

In one sentence

“A merchant’s storefront that both people and AI agents can shop from — safely, explainably, and with proof of every decision it made.”

2 Why We Are Building This

Why this track

- Track 01 explicitly asks for two things: grow merchant revenue, **or** make the merchant transactable by an AI buyer end-to-end.
- It is tied to live 2026 industry movement — NPCI’s UAP, and protocols like ACP, AP2, and x402. Agent-to-agent commerce is a real, current problem, not a made-up hackathon theme.

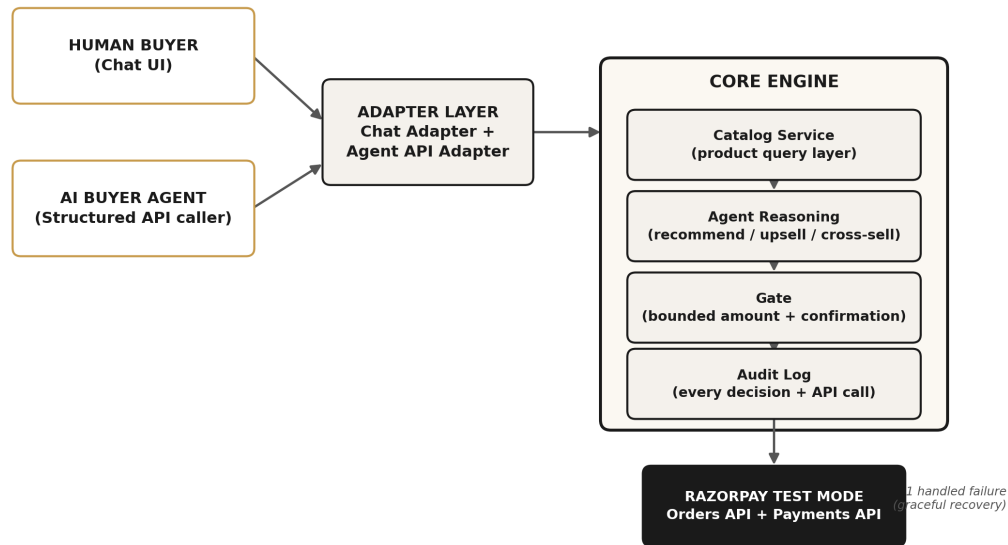
Why this specific angle

- A plain shopping chatbot is the most obvious build in the entire buildathon — most applicants will submit some version of it.
- Almost nobody will attempt the AI-buyer-agent side, because it needs real protocol/API design, not just prompt engineering.
- Building both paths on one shared engine answers the **full** track brief instead of half of it, without doubling the build time.

3 How We Are Building It

One core engine. Two thin adapters sit on top of it — one for a human via chat, one for an AI agent via a structured endpoint. The engine itself never knows or cares which one is calling it.

NEXUS — SYSTEM ARCHITECTURE



The 5 core pieces

Piece	What it does
Catalog Service	Own product data (10–20 items), exposed as a schema an LLM or agent can query. Not a Razorpay feature — built in-house.
Agent Reasoning	Decides what to recommend, when to upsell or cross-sell, and when to move to checkout.
Gate	Every order must pass three checks: amount bounded within a limit, explicit confirmation step, reasoning logged before execution.
Razorpay Integration	Orders API (create order) then Payments API (capture / fetch). Test mode only — no real money moves.
Audit Log	Every agent decision, every API call, and every result — stored and viewable. This is the proof layer for reviewers.

The 2 entry adapters

Adapter	Who uses it	What it looks like
Chat Adapter	A human buyer	Plain chat UI — type what you want, get recommendations, confirm, buy.
Agent API Adapter	A second AI agent, self-built as the demo buyer	A structured, documented endpoint — no chat, just machine-readable calls.

The one deliberate failure

One failure is intentionally injected — e.g. a payment failure or an invalid amount — and the engine must recover or escalate gracefully instead of breaking. This is a stated requirement of the track, not optional polish.

4 Model / System Architecture — Detailed

This is the layer-by-layer flow of a request through the system, from the moment a buyer (human or agent) speaks, to the moment an order is created on Razorpay.

- **Entry** — Buyer sends input: natural language (human, via Chat Adapter) or a structured call (AI agent, via Agent API Adapter).
- **Normalize** — The adapter converts input into one common internal request format, so the Core Engine sees an identical shape regardless of source.
- **Catalog Query** — Agent Reasoning queries the Catalog Service, as a callable tool, for matching or relevant products.
- **Recommend / Upsell** — Agent Reasoning selects the best match and evaluates one upsell or cross-sell opportunity — the “growth” behavior the track asks for.
- **Gate Check** — Before any money action: is the amount within the allowed bound? Has the buyer explicitly confirmed? Is the reasoning logged?
- **Order Creation** — On pass, the engine calls Razorpay's Orders API (test mode), then Payments API to capture and fetch payment status.
- **Audit Write** — Every step above — the query, the recommendation, the gate decision, the API call and its result — is written to the audit log.
- **Response** — The result returns through the same adapter the request came in on: a chat reply, or a structured response to the AI agent.
- **Failure Path** — If any step fails (e.g. payment declined), the engine catches it, logs it, and returns a graceful, explained outcome instead of crashing.

Why this design specifically

- One engine, two adapters — fully answers both halves of the track brief without duplicating logic or doubling build time.
- Gate-before-execution reuses a pattern already proven in earlier VS7 builds (explicit approval before any external action) — reused discipline, not new risk.
- The audit log is the reviewer's proof — it turns “trust me, it works” into a visible, queryable record of every decision the agent made.

5 What We Are Using — Tech Stack

Layer	Tool	Why
Backend	FastAPI (Python)	Already proven in an earlier build — async, fast to build on.
LLM / Reasoning	Groq / OpenAI / Claude, with tool-calling	Powers catalog queries plus recommendation and upsell logic.
Database	SQLite or Postgres	Stores catalog data and the audit log.
Payments	Razorpay Test Mode — Orders API + Payments API	Required by the track. No real money is involved.
Human Frontend	Simple chat UI	Minimal and functional — not the differentiator, so kept light.
AI Agent Frontend	A second, self-built toy agent	Calls the Agent API Adapter live — the standout demo moment.

6 When — 10-Day Build Timeline

Day	Focus
1	Generate Razorpay test-mode keys. Confirm the Orders API works via a live curl call. Design the catalog schema.
2–3	Build the catalog: seed 10–20 products plus a query layer. Define agent tool schema (search, get product, create order).
4	Build the Gate and the Audit Log — shared by both adapters.
5	Wire Razorpay Orders and Payments APIs into the core engine end-to-end, test mode only.
6	Build the Chat Adapter (human path).
7	Build the Agent API Adapter, plus the second toy AI agent that calls it — the standout piece.
8	Inject and handle one deliberate failure gracefully, on both paths.
9	Clean the repo, write the README, finalize the architecture diagram.
10	Record the 5-minute pitch video: a human buying, and a second AI agent transacting, same engine underneath. Submit.

7 How It Works — A Worked Example

A fictional merchant, **Northlight Outdoors** (camping gear, 15-product catalog), walks through all three paths: a human buyer, an AI buyer agent, and the one handled failure. The gate's bound for this demo is set at **Rs. 5,000** — the ceiling for an order to be auto-approved without extra friction.

Scenario A — Human Buyer

- Buyer types: “I need a good sleeping bag for winter camping, budget around Rs. 3000.”
- Chat Adapter passes this to the Core Engine. Agent Reasoning calls the Catalog Service: search for sleeping bags, winter-rated, under Rs. 3000.
- Catalog returns the Arctic Pro Sleeping Bag, Rs. 2,799.
- Agent replies with the match, and offers one upsell: a compatible sleeping mat at Rs. 499 — this is the “growth” behavior the track asks for.
- Buyer accepts both. Gate checks the total, Rs. 3,298 — under the Rs. 5,000 bound — and asks for explicit confirmation.
- Buyer confirms. Engine calls the Orders API, then the Payments API to capture the test payment.
- Audit Log records the full chain: query, recommendation, upsell offered, gate check passed, order ID, payment status.
- Buyer receives: “Order placed. Order ID RZP_test_8821.”

Scenario B — AI Buyer Agent

- A second, self-built toy agent (“ScoutBot”) is instructed: “Buy one tent under Rs. 5000 from Northlight Outdoors.”
- ScoutBot calls the Agent API Adapter directly — a structured catalog query, no chat involved.
- The adapter returns structured JSON: three matching tents with price, stock, and specs.
- ScoutBot picks the best match by its own logic and calls the order endpoint, with a reasoning field included in its request.
- The Gate applies the exact same checks as the human path: amount bounded, confirmation required, reasoning logged.
- The engine creates the order via Razorpay, identically to Scenario A.
- The Audit Log shows this transaction tagged *source: agent*, sitting right next to the human one tagged *source: chat* — same log, same schema, provably the same engine underneath.

Scenario C — The Handled Failure

- A buyer, human or agent, tries to order something priced Rs. 12,000 — above the Rs. 5,000 gate bound.
- The Gate check fails. The engine does not crash or silently reject the request.
- It responds instead: “This order exceeds the auto-approval limit. Would you like to split it into two orders, or reduce the quantity?”
- The Audit Log records the rejected attempt and the reason — visible and explainable to a reviewer, not hidden.

8 Final Output — What This Produces

By Day 10, the finished submission includes

- **A public GitHub repo** — working code, a clean README, sensible commit history.
- **A live demo** — one human buyer flow (chat, recommend, upsell, confirm, order created) and one AI-agent buyer flow (agent calls the API, order created), both hitting Razorpay test mode.
- **An audit trail** — a viewable log of every decision and API call made during both demo runs.
- **One handled failure** — shown live, recovered gracefully, not hidden.
- **An architecture diagram** — the same shape as the one in Section 3.
- **A 5-minute pitch video** — walks through the problem, the architecture, both buyer paths live, and the failure-recovery moment.

What it proves to reviewers

- The build answers the **entire** track brief — the human growth path and AI-buyer transactability — not half of it.
- Every money action is explainable, bounded, and gated: exactly the stated bar.
- It is infrastructure-minded, not just a chatbot — a rarer, more memorable submission in a pool where most entries will look alike.

This document is the locked scope. No feature outside the 5 core pieces and 2 adapters gets added during the 10-day build.